



Applied Cryptography

Summer 2026

Part 2: Real-World Cryptography

2.5: High-Assurance Cryptography

Nadim Kobeissi

<https://appliedcryptography.page>

Acknowledgments

Part 1 of this slide deck is based on:

Manuel Barbosa, Gilles Barthe, Karthikeyan Bhargavan, Bruno Blanchet, Cas Cremers,
Kevin Liao and Bryan Parno

SoK: Computer-Aided Cryptography

IEEE Symposium on Security and Privacy, 2021

<https://appliedcryptography.page/paper/#sok-verif>

Part 2 is based on new research by the instructor.

Section 1

Introduction

Securely implementing cryptography

- Designing, implementing, and deploying cryptographic mechanisms is notoriously hard
- Regular discoveries of high-profile design flaws, devastating implementation bugs, and side-channel vulnerabilities
- Even in **widely deployed** mechanisms!

Three levels of complexity:

1. **Design Level:** Security proofs, adversarial models
2. **Implementation Level:** Correctness, performance, memory safety
3. **Deployment Level:** Side-channel resistance

The problem

Attackers can break designs, exploit bugs, recover secrets via side-channels, or **any combination**

- **Pen-and-paper proofs:** Consider only “cores,” still error-prone
- **Aggressive optimization:** Increases bug risk
- **Current modus operandi:** Select few experts with rudimentary tools

This approach **cannot keep pace** with the rate of innovation in cryptography

Enter: Computer-Aided Cryptography (CAC)

- Active area of research: **formal, machine-checkable** approaches to designing, analyzing, and implementing cryptography
- **Design level:** Tools manage complexity of security proofs, can reveal subtle flaws
- **Implementation level:** Guarantee correct behavior, even for highly optimized code
- **Deployment level:** Check side-channel protections

Individual tools may only address *part* of the problem, but when **combined**, they provide a **high degree of assurance**

Success stories

- Computer-aided security analyses were influential in **TLS 1.3** standardization
- **HACL[★] library**: Integrated into Mozilla Firefox's NSS
- **Fiat Cryptography**: Elliptic curve code in Google's BoringSSL
- **EverCrypt**: Routines used in Linux kernel's Zinc crypto library

Formally verified code at Internet scale!

Missing: Real-world software complexity

- Pen-and-paper proofs consider **isolated algorithms**
- Real deployments involve massive software stacks:
 - Operating system layers
 - Network protocol implementations
 - Library dependencies
 - Hardware abstraction layers
- Each layer introduces new attack surfaces and unexpected interactions
- **Result:** Security proofs may not hold in practice!

Challenges ahead

- CAC risks becoming a **victim of its own success**
- Trust can be undermined by difficulty understanding guarantees and fine-print caveats
- Field is increasingly broad, complex, and rapidly evolving
- **Post-quantum transition:** New algorithms need verification
- **Scaling:** Moving from primitives to whole cryptographic systems

No one has complete understanding of every facet

Section 2

Design-Level Security

Symbolic security: Example

Symmetric encryption modeled by:

- Functions: Enc and Dec
- Equational theory:

$$\text{Dec}(\text{Enc}(m, k), k) = m$$

- Decrypting ciphertext $\text{Enc}(m, k)$ with key k recovers plaintext m
- Adversary can only use specified primitives and equations
- Can decrypt only if it has the **entire** secret key

Two flavors of design-level security

- Developed over years in **two separate communities**:
- **Symbolic Security** (Formal methods community)
 - Messages are abstract **symbols**, not bits
 - Cryptographic primitives are **perfect black boxes**
 - Can only decrypt with the **exact** key
- **Computational Security** (Cryptography community)
 - Messages are **bitstrings** of specific length
 - Encryption is a **probabilistic algorithm**
 - Adversaries have **bounded computational power**

Symbolic vs. Computational: Key differences

Aspect	Symbolic	Computational
Messages	Abstract terms	Bitstrings
Adversary	Rule-based	Probabilistic algorithm
Security	All-or-nothing	Probabilistic bounds
Analysis	Often decidable	Generally undecidable
Guarantees	"Cannot break"	"Hard to break"

- **Symbolic:** Quick protocol flaw detection, automated analysis
- **Computational:** Concrete security guarantees, cryptographic proofs
- **Both:** Different insights, complementary verification

Symbolic security properties

Two main flavors:

1. Trace properties

- Bad events never occur on any execution trace
- Example: Secret data never in adversarial knowledge

2. Equivalence properties

- Adversary cannot distinguish between two protocols
- Example: Cannot differentiate real secret from random value

Equivalence properties cannot be naturally expressed as trace properties

Computational security properties

Characterized along two axes:

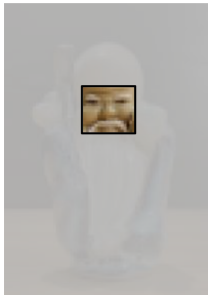
Type	Quantification
Game-based	Concrete
Simulation-based	Asymptotic

- CAC tools primarily focus on **game-based, concrete security**
- Simulation-based proofs: More complex, support composition
- Concrete security implies asymptotic security

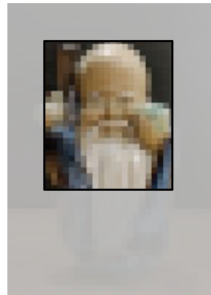
How protocol models end up, intuitively



Reality



Computational



Symbolic

Source: Cas Cremers

Why design-level security matters

- Mathematical arguments are the **only way** to demonstrate security against entire classes of attacks
- Security proofs are **necessary** for any new standard

But pen-and-paper arguments are alarmingly fragile:

1. **Erroneous arguments:** Tedious, error-prone, errors go unnoticed for years
2. **Inappropriate modeling:** Simplified models miss real attacks

CAC tools can detect flaws and manage complexity beyond pen-and-paper reach

Key features of symbolic analysis tools

- **Unbounded sessions:** Can analyze arbitrary number of concurrent sessions, proving **absence of attacks** in the model
- **Trace properties:** Bad events never occur on any execution trace
- **Equivalence properties:** Adversary cannot distinguish between two protocols
- **Equational theories:** Model algebraic properties of crypto operations (XOR, DH, etc.)
- **Global mutable state:** Support for shared databases, key servers

Modern unbounded tools often **outperform** bounded tools, even for small session counts!

Symbolic analysis tools overview

Tool	Unbound	Trace	Equiv	Eq-thy	State	Link
CPSA [▷] [16]	●	●	○	○	●	○
F7 [◇] [17]	●	●	○	●	●	●
↳F5 [◇] [18]	●	●	○	●	●	●
Maude-NPA [▷] [19]	●	●	● ^d	●	○	○
ProVerif ^{*†} [20]	●	●	● ^d	●	○	○
↳fs2pv ^{◇†} [21]	●	●	○	●	○	●
↳GSVerif ^{*†} [22]	●	●	○	●	●	○
↳ProVerif-ATP ^{*†} [23]	●	●	○	●	○	○
↳StatVerif ^{*†} [24]	●	●	● ^d	●	●	○
Scyther [▷] [25]	●	●	○	○	○	○
scyther-proof ^{▷‡§} [26]	●	●	○	○	○	○
Tamarin ^{*‡} [27]	●	●	● ^d	●	●	○
↳SAPIC [*] [28]	●	●	○	●	●	○
CI-AtSe [▷] [29]	○	●	○	●	●	○
OFMC ^{▷†} [30]	○	●	○	●	●	○
SATMC [▷] [31]	○	●	○	○	●	○
AKISS [*] [32]	○	○	● ^t	●	●	○
APTE [*] [33]	○	○	● ^t	○	●	○
DEEPSEC [*] [34]	○	○	● ^t	●	●	○
SAT-Equiv [*] [35]	○	○	● ^t	○	○	○
SPEC ^{*,§} [36]	○	○	● ^o	○	○	○

Source: Manuel Barbosa, Gilles Barthe, Karthikeyan Bhargavan, Bruno Blanchet, Cas Cremers, Kevin Liao and Bryan Parno,

SoK: Computer-Aided Cryptography, IEEE Symposium on Security and Privacy, 2021.

ProVerif: Real-world success stories

TLS 1.0: Verification in seconds to hours

TLS 1.3 (part of my Ph.D. thesis!): ~1 hour

Signal Protocol (part of my Ph.D. thesis!): Minutes to days

Noise Protocol Framework (also part of my Ph.D. thesis!): Seconds to days

- Demo: <https://noiseexplorer.com>

Key insight: More Diffie-Hellman operations = longer analysis

Tamarin: Tackling massive protocols

5G Authentication Key Exchange: ~5 hours

TLS 1.3: ~1 week, 100GB RAM!

DNP3 SAv5 (power grid security): Several minutes

Why ProVerif and Tamarin succeed:

- Extensive documentation and tutorials
- Library of case studies
- Practical features: Tamarin has a GUI, ProVerif reconstructs attacks in HTML

Computational analysis tools overview

Tool	RF	Auto	Comp	CS	Link	TCB
AutoG&P [◇] [55]	●	●	○	●	○	self, SMT
CertiCrypt ^{▷◇} [56]	●	○	○	●	●	Coq
CryptHOL [◇] [57]	●	○	●	●	○	Isabelle
CryptoVerif ^{*◇} [58]	●	●	○	●	●	self
EasyCrypt ^{▷◇} [59]	●	○	●	●	●	self, SMT
F7 [◇] [17]	●	○	●	○	●	self, SMT
F ^{*◇} [60]	●	○	●	○	●	self, SMT
FCF [◇] [61]	●	○	●	●	●	Coq
ZooCrypt [◇] [62]	●	●	○	●	○	self, SMT
Reasoning Focus (RF)		Concrete security (CS)		Specification language		
● – automation focus		● – security + efficiency		★ – process calculus		
● – expressiveness focus		● – security only		▷ – imperative		
		○ – no support		◇ – functional		

Source: Manuel Barbosa, Gilles Barthe, Karthikeyan Bhargavan, Bruno Blanchet, Cas Cremers, Kevin Liao and Bryan Parno,

SoK: Computer-Aided Cryptography, IEEE Symposium on Security and Privacy, 2021.

Game-based security

- Probabilistic experiment: “game” between challenger and adversary
- Security statement:
“For all adversaries, probability of achieving goal $\leq$ threshold”
- Core methodology: **Game hopping**
 1. Start with original game
 2. Transform step-by-step
 3. Reach game where success probability is known
 4. Bound probability increases at each step
 5. Similar to what we saw in Part 1 of this course!

Key features of computational analysis tools

Automation vs. Expressivity: Every tool makes this tradeoff

- Automated tools: Push-button proofs, limited patterns
- Expressive tools: Arbitrary arguments, more manual effort

Other key dimensions:

- **Composition support:** Can we reason about pieces separately?
- **Concrete security bounds:** Not just “secure” but “how secure”
- **Link to implementation:** Can guarantees reach real code?
- **Trusted computing base:** Theorem prover? Tool itself? SMT solver?

Smaller TCB = Higher confidence

CryptoVerif: Automated protocol analysis

Best for highly automated protocol analysis

- Both proof-finding and proof-checking
- Excellent for key exchange protocols
- Applied π -calculus input language

Success stories:

- TLS 1.3 (part of my Ph.D. thesis!)
- Signal Protocol (part of my Ph.D. thesis!)
- WireGuard VPN

F[★]: Beyond the cryptographic core

Best for analyzing full protocols and systems

- General-purpose verification language with **dependent types**
- A well-typed program is a **proof of its specification**
- Type checking = Proof checking, with Z3 doing the heavy lifting!

Example:

- Type: `point : {(x,y) : int * int | y*y = x*x*x + a*x + b}`
- This type **only** accepts valid elliptic curve points
- Your code literally cannot process invalid points if it type-checks

F[★]: Real-world impact

How F[★] handles computational proofs:

1. Start with detailed protocol description
2. Transform into ideal program with ideal crypto functionalities
3. Validate transformation manually, verify final program automatically

Success stories:

- Discovered new attacks on TLS
- miTLS: Verified reference implementation of TLS 1.2
- Implemented and verified TLS 1.3 record layer

EasyCrypt successes: AWS Key Management System, SHA-3 standard

The fine print

Computer-aided security proofs are only as good as the statements being proven

- Understanding these statements can be **challenging**
- Most proofs rely on **implicit assumptions**
- Reconstructing top-level statements is hard, even for experts
- Tools may have bugs (like any software)

We will return to this theme in Part 2...

Section 3

Functional Correctness and Efficiency

The fundamental tension

Design-level security is not enough!

We need implementations that are **correct**, **fast**, and **memory safe**

Easy path: Reference implementation

- ✓ Straightforward correctness
- ✗ Terrible performance

Hard path: Optimize aggressively

- ✓ Blazing fast
- ✗ Correctness nightmare

How far will developers go for speed?

OpenSSL's approach:

1. Write Perl code...
2. ...that generates strings of text...
3. ...that other Perl scripts interpret...
4. ...to produce C preprocessor input...
5. ...which generates platform-specific assembly!

This is one of the *fastest* crypto libraries

But imagine verifying *that*

The language dilemma

High-level languages? Too slow.

Reality of high-performance crypto:

- C for “portable” performance
- Assembly for ultimate speed
- Platform-specific optimizations everywhere

The cost:

- Memory safety becomes critical concern
- Bugs can leak secrets (Heartbleed)
- Verification becomes much harder

Best-in-class crypto needs manual optimization; compilers can't match expert assembly

Even experts make mistakes

A handful of experts write all high-speed crypto

But even they get it wrong:

- OpenSSL AES-GCM optimization → message forgeries
- OpenSSL arithmetic bug → full key recovery
- 24 vulnerabilities in libcrypto (2016–2019)
- Heartbleed: Memory safety bug leaked secrets

Traditional QA is inadequate:

- Auditing: Expensive, can miss subtle bugs, doesn't scale
- Testing: Cannot cover the huge cryptographic input space

Enter program verification

Cryptographic code is an **ideal target** for verification

Why? Critically important, relatively small codebase, well-defined specs

Two approaches to correctness:

1. **Equivalence to reference:** Same outputs for same inputs
2. **Functional specification:** Pre/post-conditions on inputs and outputs

The compiler gap:

- Most verification happens at source level, but we execute machine code
- Solution: Verified compilers (CompCert) or assembly-level verification

Real deployment success stories

HACL★ in Firefox's NSS:

- Replaced entire ciphersuite
- Performance within 5% of OpenSSL
- Professional packaging
- Perfect timing: post-Heartbleed trust crisis

Three critical success factors:

1. Performance parity
2. API compatibility (drop-in)
3. Industry collaboration

Fiat Crypto: Powers Google's BoringSSL

Billions of users rely on verified crypto every day

Input languages: Many flavors

Verification-oriented languages:

- **Dafny**: High-level imperative
- **F[★]**: Functional with effects
- **Gallina** (Rocq): Pure functional

Assembly-like languages:

- **CryptoLine**: Low-level verification
- **Jasmin**: Assembly with high-level features
- **Vale**: Structured assembly

Source-level verification needs a verified compiler for guarantees;
assembly-level verification gives direct machine-code guarantees

Curve25519: Verified implementations win

C implementations:

- **hacl** (verified) → Beats donna64 (fast unverified C)
- **fiat** (verified) → Beats donna64
- Multiple verified implementations faster than best unverified C

Assembly implementations:

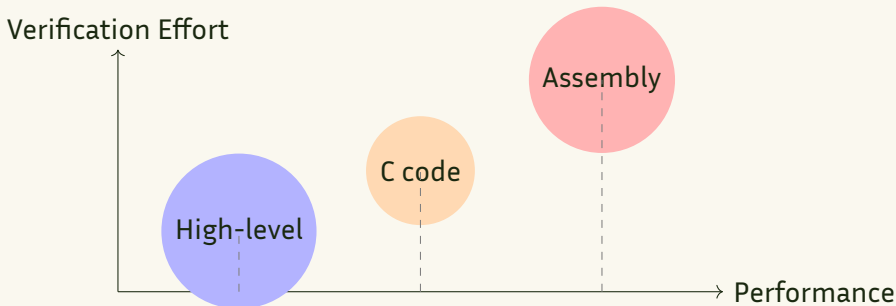
- **evercrypt** (verified) → Matches fastest unverified

Old wisdom: Verified code = Slow code

New reality: Verified code matches or beats unverified!

No conceptual barrier to verifying the fastest implementations

The verification effort spectrum



Higher performance = Higher verification effort
But even at the assembly level, verification is now practical

Section 4

Implementation-Level Security

Side channels: The hidden attack surface

Design-level security assumes attackers only see inputs and outputs.

Reality: implementations leak much more!

Information leaks through:

- **Timing:** How long operations take
- **Memory access:** Which cache lines are touched
- **Power consumption, EM radiation, acoustics**

Real attacks: RSA/AES key recovery, Lucky 13, cross-VM extraction

The constant-time solution

Make execution independent of secrets

Rules: No secret-dependent branches, no secret-dependent memory access, no variable-time operations on secrets

Example — constant-time selection:

Insecure: `if (secret_bit) { result = a; } else { result = b; }`

Secure: `mask = -secret_bit; result = (a & mask) | (b & ~mask);`

Same result, no branches, *much harder to understand*

Compilers and hardware betray you

The compiler problem:

- Compiler “optimizations” can reintroduce branches, use lookup tables, short-circuit operations
- The compiler doesn’t know about your security requirements

The hardware problem:

- Branch prediction, speculative execution, cache hierarchies
- Variable-latency instructions
- Spectre/Meltdown-style attacks

Even perfect constant-time source code may not survive to execution!

Why constant-time is hard

Natural code patterns become forbidden

You can't write:

1. `if (secret_bit) { ... }`
2. `array[secret_index]`
3. Return early if `secret ? = 0`
4. Use division only when `secret  $\neq$  0a`

Instead, you must:

- Use bit manipulation tricks
- Process all data uniformly
- Avoid natural optimizations

^a<https://kyberslash.cr.yp.to>

The TLS patching cycle

Even experts get constant-time wrong

The vicious cycle:

1. Original TLS timing vulnerability discovered
2. Developers patch it
3. The patch itself creates Lucky 13
4. Lucky 13 gets patched
5. The Lucky 13 patch introduces a *new* timing vulnerability!

Each “fix” introduced new vulnerabilities!

Formal leakage models

Three levels of increasing strictness:

1. **Program counter (PC) model:** Leaks control flow only
 - Protects against simple timing attacks
2. **Constant-time model:** Leaks PC + memory addresses
 - Protects against cache and timing attacks
 - Most common in practice
3. **Size-respecting model:** Leaks PC + memory + operand sizes
 - Protects against variable-time instruction attacks

Security = Leakage independent of secrets (**observational non-interference**)

Example: Constant-time selection

Insecure (natural) code:

```
if (secret_bit) {  
    result = a;  
} else {  
    result = b;  
}
```

Secure (constant-time) code:

```
mask = -secret_bit; // 0x00...0 or 0xFF...F  
result = (a & mask) | (b & ~mask);
```

Same result, no branches, harder to understand!

CAC for implementation security

What if we could **prove** constant-time?

CAC tools can:

- Automatically check constant-time compliance for all possible inputs
- Find subtle violations
- Even repair non-compliant code
- Provide formal guarantees

Important caveats:

- Proofs are relative to the leakage model
- Hardware may leak more than modeled (e.g., Spectre)
- Physical attacks bypass all software protections

Implementation verification tools overview

Implementation(s)	Target(s)	Tool(s) used	Computational security	Functional correctness	Efficiency	Side-channel resistance
RSA-OEAP [172]	C	EasyCrypt, Frama-C, CompCert	●	●	○	●
Curve25519 scalar mult. loop [114]	asm	Coq, SMT	—	●	●	○
SHA-1, SHA-2, HMAC, RSA [131]	asm	Dafny, BoogieX86	—	●	●	○
HMAC-SHA-2 [173]	C	FCF, VST, CompCert	●	●	○	○
MEE-CBC [174]	C	EasyCrypt, Frama-C, CompCert	●	●	○	●
Salsa20, AES, ZUC, FFS, ECDSA, SHA-3 [175]	Java, C	Cryptol, SAW	○	●	●	○
Curve25519 [176]	OCaml	F*, Sage	—	●	○	●
Salsa20, Curve25519, Ed25519 [102]	asm	Jasmin	○	○	●	●
SHA-2, Poly1305, AES-CBC [103]	asm	Vale	○	●	○	●
HMAC-DRBG [177]	C	FCF, VST, CompCert	●	●	○	○
HACL* ¹ [5]	C	F*	●	●	●	●
HACL* ¹ [5]	C	F*, CompCert	●	●	○	●
HMAC-DRBG [178]	C	Cryptol, SAW	—	●	○	○
SHA-3 [69]	asm	EasyCrypt, Jasmin	●	●	●	●
ChaCha20, Poly1305 [117]	asm	EasyCrypt, Jasmin	○	●	●	●
BGW multi-party computation protocol [179]	OCaml	EasyCrypt, Why3	●	●	○	○
Curve25519, P-256 [6]	C	Fiat Crypto	—	●	●	○
Poly1305, AES-GCM [104]	asm	F*, Vale	○	●	●	●
Bignum code ⁴ [98]	C	CryptoLine	—	●	●	○
WHACL* ¹ , LibSignal* [180]	Wasm	F*	●	●	●	●
EverCrypt ² [7]	C	F*	○	●	●	●
EverCrypt ³ [7]	asm	F*, Vale	○	●	●	●

Computational security	Functional correctness	Efficiency	Side-channel resistance
● – verified	● – target-level	● – comparable to asm ref	● – target-level
● – partially verified	● – source-level	● – comparable to C ref	● – source-level
○ – not verified	○ – not verified	○ – slower than C ref	○ – not verified
— – not applicable			

¹(ChaCha20, Salsa20, Poly1305, SHA-2, HMAC, Curve25519, Ed25519) ²(MD5, SHA-1, SHA-2, HMAC, Poly1305, HKDF, Curve25519, ChaCha20)
³(AES-GCM, ChaCha20, Poly1305, SHA-2, HMAC, HKDF, Curve25519, Ed25519, P-256) ⁴(In NaCl, wolfSSL, OpenSSL, BoringSSL, Bitcoin)

Source: Manuel Barbosa, Gilles Barthe, Karthikeyan Bhargavan, Bruno Blanchet, Cas Cremers, Kevin Liao and Bryan Parno, SoK: Computer-Aided Cryptography, IEEE Symposium on Security and Privacy, 2021.

Hardware attacks: TEE.fail (2025)

Physical interposition on DDR5 bus:

- Built with cheap, off-the-shelf electronics
- Inspects all memory traffic in real-time

Devastating impact:

- Breaks Intel TDX
- Breaks AMD SEV-SNP
- Extracts attestation keys
- Compromises Nvidia GPU Confidential Computing

Hardware-level attacks bypass all software protections

<https://tee.fail/>

Section 5

Case Study: Lessons from TLS

TLS before 1.3: Reactive patching

One of the most important deployments of cryptography
Powers HTTPS, secures billions of connections daily

The endless loop (pre-1.3):

1. Attack found → Quick patch → Long-term fix → New attack

When academics looked at real TLS: BEAST, CRIME, Lucky 13, POODLE, Logjam...

No substantial academic analysis during design

TLS 1.3: A paradigm shift

From reactive patching to proactive design

- Academic community consulted **during** design: 28 drafts over 4+ years
- Multiple CAC tools applied: ProVerif, Tamarin, CryptoVerif, F★
- Real attacks found and fixed **before** standardization:
 - Draft 10: Client impersonation in PSK-resumption
 - Draft 13: Attack on 0-RTT client authentication
 - Draft 21: Unexpected authentication behavior

Three lessons from TLS 1.3

Lesson 1: Formal methods find real flaws

- Just *trying* to write formal specs uncovers ambiguities and corner cases
- *"If you can't specify it clearly, it's probably broken"*

Lesson 2: Protocols are moving targets

- 28 drafts = 28 different protocols to verify
- Machine-checked proofs can evolve with the protocol; manual proofs can't keep pace

Lesson 3: Design for verifiability

- Small changes (key separation, consistent tagging) dramatically simplify analysis
- Negligible performance impact, huge verification benefit

Impact beyond TLS

TLS 1.3 proved formal methods can scale

The new playbook for protocol design:

1. Involve formal methods experts early
2. Design with verification in mind
3. Use tools throughout development
4. Update proofs with each draft
5. Fix issues before standardization

From “ship and patch” to “verify and ship”

This is the future of secure protocol design

Part 2

Verification Theatre: Critically Examining High-Assurance Cryptography

Based on: Nadim Kobeissi, *Verification Theatre: False Assurance in Formally Verified Cryptographic Libraries*, 2026.

Section 6

The Verification Boundary

What is a verification boundary?

Every formally verified system embeds a **verification boundary**

- The interface between code with **machine-checked proofs** and code that is **trusted without them**
- Outside the boundary: compilers, OS, hardware, platform intrinsics, “glue code”
- The existence of this boundary is well understood...

But it is not always communicated clearly

Good boundary documentation exists

- **CompCert**: Explicitly documents which compiler passes are verified and which (parser, assembler, linker) are not
- **seL4**: Maintains a dedicated page listing every trust assumption about hardware behavior
- **EverCrypt**: Explicitly identifies compiler, Vale embedding semantics, and hardware as trusted components
- **Jasmin**: Documents which compiler passes have machine-checked proofs

These projects treat verification boundaries as **first-class documentation**

When boundaries go wrong

What happens when a library is marketed as “formally verified”
without qualification?

- Users reasonably interpret this as a comprehensive guarantee
- The gap between verification *claims* and verification *reality* creates false assurance
- We call this gap “**verification theatre**”

Deploying formal methods with unqualified claims creates an appearance of correctness while leaving significant code paths unverified

Case study: Cryspen's libcrux

- Cryspen: “services and software for high assurance software”
- libcrux repository: “the formally verified crypto library”
- Documentation: “every implementation in libcrux is formally verified”

Real-world deployment:

- ML-KEM implementation used internally at Google
- hpke-rs is a dependency of Signal's `signal-crypto` crate
- hpke-rs used by OpenMLS

I found **13 vulnerabilities** that escaped formal verification

The 13 vulnerabilities at a glance

ID	Description	Component
V1	Incorrect ARM64 SHA-3 intrinsic	libcrux ML-KEM
V2	Cross-backend endianness	libcrux ML-KEM
V3	Missing X25519 zero check	hpke-rs
V4	Nonce reuse via u32 overflow	hpke-rs
V5	ECDSA missing low-S	libcrux ECDSA
V6	Ed25519 double clamping	libcrux Ed25519
V7	AES-GCM decrypt panic	libcrux-psq
V8	ML-DSA dead norm check	libcrux ML-DSA
V9	ML-DSA hint bound skip	libcrux ML-DSA
V10	Wrong decompression constant	ML-KEM spec
V11	Missing inverse NTT	ML-KEM spec
V12	False serialization proof	ML-KEM proof
V13	Wrong multiplication spec	ML-DSA spec

Verification status reality

ML-KEM verification status as documented by Cryspen

Backend	Module	Lax	Safety	Correct.
Generic	const_time, hash, ...	✓	✓	✓
	ntt/invert_ntt	✓	✓	—
	matrix/serial./sampl	✓	—	—
Portable	arith/ntt/serial....	✓	✓	✓
	sampling	✓	✓	—
AVX2	arith/ntt/serial.	✓	✓	✓
	compr/sampling	✓	✓	—
NEON	<i>all modules</i>	✓	—	—

The NEON backend—implicated in a platform-dependent output failure—has **no completed proofs** for any module beyond lax checking

Section 7

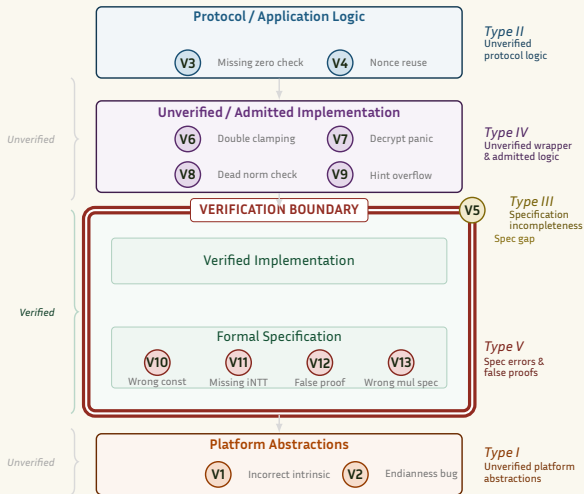
A Taxonomy of Verification Failures

Five types of verification boundary failure

Type	Category	Vulns
I	Unverified platform abstractions	V1, V2
II	Unverified protocol logic	V3, V4
III	Specification incompleteness	V5
IV	Unverified wrapper logic	V6, V7, V8, V9
V	Spec errors & false proofs	V10–V13

- Types I–IV: Bugs at or **beyond** the verification boundary
- Type V: Bugs **inside** the formally verified code itself
- Type V is the most insidious: it corrupts the foundation

The verification boundary, visually



The formal verification pipeline

How does Cryspen's verification work?

1. **Specification:** Algorithm spec written in hacspect (Rust subset) or F^*
2. **Implementation:** Optimized Rust with multiple backends (portable, AVX2, NEON)
3. **Extraction and Proof:** hax toolchain extracts Rust to F^* ; provers verify correctness, safety, and secret independence
4. **Compilation:** Verified source compiled by **unverified** compiler (LLVM/rustc)

Each stage introduces a trust boundary

Section 8

Type I: Unverified Platform Abstractions

Type I: Unverified Platform Abstractions

The verified code depends on platform intrinsics treated as **axioms**

- Formal proofs **assume** these operations implement their specified semantics
- The actual implementations may contain bugs
- Marked `#[hax_lib :: opaque]`: verifier accepts specification without proof

V1: Incorrect ARM64 SHA-3 intrinsic

- The `_vxarq_u64` intrinsic wrapper for ARM64 SHA-3
- On platforms **without** SHA-3 hardware support, a fallback path is used
- The fallback implementation was **incorrect**
- Produced wrong cryptographic output on certain ARM64 devices

Why verification missed it:

- Function marked as opaque to the verifier
- Verifier accepted the *specification* as an axiom
- The axiom was correct; the *implementation* was wrong
- Discovered independently by Filippo Valsorda

V2: Cross-backend endianness bug

- ML-KEM incremental API: `from_bytes/to_bytes` serialization
- Portable backend used **different byte order** than SIMD backends
- Serialize state on one platform, deserialize on another → **failure**

Real-world impact:

- Caused **decryption failures in Signal's post-quantum ratchet**
- Reported by GuuJiang (December 2025)
- Each backend was individually correct, but they were **mutually incompatible**

Verification proved each backend correct in isolation; the cross-backend property was never specified

Type I: Lessons

What Type I failures teach us

- Platform abstractions are **axioms**: the verifier takes them on faith
- Cross-platform properties (endianness, representation) are rarely specified
- Testing each backend individually is **not enough**
- Need cross-backend compatibility testing
- Axiomatized specifications should be **proved or exhaustively tested** against a reference

Section 9

Type II: Unverified Protocol Logic

Type II: Unverified Protocol Logic

Verified primitives used correctly...
but the protocol logic around them has bugs

V3: Missing X25519 zero check

- RFC 9180 **requires** checking if DH output is all-zero
- hpke-rs returns the raw result without validation
- Small-subgroup attack: adversary can force shared secret to zero

V4: Nonce reuse via integer overflow

- HPKE sequence counter is u32 (wraps at 2^{32})
- In release builds, wrapping is **silent**: counter goes back to 0
- After 2^{32} messages: **nonce reuse!**

Type II: Lessons

What Type II failures teach us

- Verifying the **crypto primitive** does not verify the **protocol**
- Standard-mandated checks (RFC 9180 zero check) live in protocol logic
- Integer overflow behavior depends on build mode (debug vs. release)
- Protocol-level code needs its own analysis—symbolic or otherwise

*The most dangerous bugs live in the space **between** verified components*

Section 10

Type III: Specification Incompleteness

Type III: Specification Incompleteness

Code is **provably correct** per its spec...
but the spec doesn't capture all real-world requirements

V5: ECDSA missing low- S normalization

- ECDSA signatures exhibit malleability: both (r, s) and $(r, n - s)$ are valid
- Bitcoin, Ethereum, and other systems **require** $s \leq n/2$ ("low- S ")
- libcrux's F[★] specification doesn't include this requirement
- Implementation is provably correct per spec, but **rejected by practical systems**

The spec was incomplete, not the proof

Section 11

Type IV: Unverified Wrapper Logic

Type IV: Unverified Wrapper Logic

The verified core is correct, but the **API layer** introduces defects

V6: Ed25519 double clamping

- Key generation wrapper applies clamping **before** hashing
- Should be: hash first, then clamp the result
- Reduces entropy of generated keys

V7: AES-GCM decryption panic

- Decryption wrapper calls `.unwrap()` on the result
- If decryption fails (e.g., tampered ciphertext): **program panics**
- Denial-of-service via any invalid ciphertext
- The correct error handling was *commented out* in the source

V8: ML-DSA norm check rendered dead code

FIPS 204 requirement: Reject signature if signer response exceeds $\gamma_1 - \beta$

The bug:

```
if vector_infinity_norm_exceeds(response, (2 << GAMMA1_EXPONENT) -  
BETA)
```

- $2 \ll \text{GAMMA1_EXPONENT}$ computes $2\gamma_1$, not γ_1
- The bound is **doubled**: $2\gamma_1 - \beta$ instead of $\gamma_1 - \beta$
- Since the signer response is already bounded by $\gamma_1 - \beta$, the check **always passes**
- A FIPS 204 security check is effectively **dead code**

libcrux accepts non-conforming ML-DSA signatures

V9: ML-DSA hint deserialization off-by-one

FIPS 204 requirement: Check that hint count does not exceed maximum

The bug: The overflow check compares against the **previous** iteration's count, not the current one

```
if (previous_true_hints_seen > max_ones_in_hint) {  
    malformed_hint = true;  
}
```

- Should check `current_true_hints_seen > max_ones_in_hint`
- The final iteration's bound is **never validated**
- Allows non-conforming signatures to pass verification

Types I–IV: Summary

Nine vulnerabilities outside the verified core

- **Real-world impact:** Signal PQ ratchet failures (V2), potential DoS (V7), non-conforming ML-DSA signatures (V8, V9)
- **Common pattern:** The verified primitive works correctly, but everything *around* it is unverified
- **Key insight:** Formal verification of primitives provides a **false sense of security** when the surrounding code is not analyzed

But the most surprising findings are still ahead...

Section 12

Type V: Specification Errors and False Proofs

Type V: The most insidious failures

Bugs **inside** the formally verified code itself

- The formal **specification** is wrong
- Or the **proofs** contain false claims silently accepted by the build system
- These undermine the **entire foundation** on which all verification rests
- Even complete proofs only establish correctness with respect to the **wrong definition**

Four concrete examples in ML-KEM and ML-DSA...

How can bugs exist inside “verified” code?

Three mechanisms that allow false proofs to survive

1. **Wrong specifications:** The formal spec is a mathematical statement—if the statement is wrong, proving it correct doesn’t help
2. **Admitted proofs:** The build system can skip proof checking entirely while reporting success
3. **False axioms:** Unproved assumptions can contradict reality; all downstream proofs become unsound

Let’s see each mechanism in action...

V10: Wrong decompression constant

FIPS 203 specification:

$$\text{decompress}_d(x) = \lceil x \cdot q/2^d \rceil = (x \cdot 3329 + 2^{d-1})/2^d$$

libcrux F[★] specification:

```
let decompress_d d x = (x * v_FIELD_MODULUS + 1664) / pow2 d
```

- $1664 = (q - 1)/2 = (3329 - 1)/2$, a fixed value unrelated to d
- The correct rounding constant should be 2^{d-1} , which varies with d
- ML-KEM uses $d \in \{1, 4, 5, 10, 11\}$: all five values produce wrong results

The specification disagrees with the standard it claims to formalize

V11: Missing inverse NTT

ML-KEM encryption computes:

- $\mathbf{u} = \text{NTT}^{-1}(\hat{A}^T \circ \hat{r}) + e_1$ (correct in spec)
- $v = \text{NTT}^{-1}(\hat{t}^T \circ \hat{r}) + e_2 + \lceil q/2 \rceil \cdot m$ (FIPS 203)

libcrux F[★] specification for v :

```
let v = poly_add (poly_add (vector_dot_product_ntt t_as_ntt  
r_as_ntt) error_2) mu
```

- The result of `vector_dot_product_ntt` is in the **NTT domain**
- But `error_2` and `mu` are in the **coefficient domain**
- Adding them together is mathematically **nonsensical**
- The missing `poly_inv_ntt` call makes the spec wrong
- Yet it **type-checks** because both are just polynomial types

V12: A false serialization proof

The claim:

```
let deserialize_12_bit_vec_lemma_bounded (v: t_Array u8 (sz 24))  
  : squash (forall (i: nat {i < 16}).  
    bounded (Seq.index result.f_elements i) 1)
```

- This claims that 12-bit deserialized values are bounded by $2^1 = 2$
- The correct bound should be $2^{12} = 4096$
- This is **obviously false**: 12-bit values can be up to 4095

How does a false proof get accepted?

The lax-mode escape hatch

The proof tactic:

```
let prove_bit_vector_equality' () : Tac unit =  
  if lax_on ()  
  then iterAll tadmit  
  else prove_bit_vector_equality'' ()
```

Three-layer failure mechanism:

1. The Makefile **admits this module by default** (ADMIT_MODULES)
2. When admitted, `lax_on()` returns true
3. The tactic calls `tadmit`: **all proof goals are accepted unconditionally**

*A false proof becomes a “verified” proof whenever the module is admitted
Full verification is **never triggered** in any automated pipeline*

V13: Wrong multiplication specification

ML-DSA AVX2 intrinsics specification:

```
let i16_mul_32extended (x y: i16): i32 =  
  (cast x : i32) *! (cast x : i32)    // should be: cast y
```

- The second parameter y is **completely ignored**
- The body computes x^2 instead of $x \cdot y$
- This specification is **axiomatized**: marked `[@@ "opaque_to_smt"]`
- The axiom is **never proved** against a reference model
- All proofs building on this axiom are **unsound**

The verified ML-DSA AVX2 proofs prove properties about x^2 , not $x \cdot y$

Why Type V bugs survive

Why do specification errors and false proofs persist?

1. **Build system admits proofs by default:** ADMIT_MODULES in Makefile
2. **Lax-mode escape hatches:** Tactics silently accept all claims
3. **Axioms are never proved:** `val` declarations without implementations
4. **Full verification never triggered:** Not in CI, not on any automated pipeline
5. **Manual status documents:** Can claim verification that doesn't exist

Specifications can contain mathematical errors, proofs can make false claims, axioms can contradict their definitions—**all simultaneously**

Type V: Summary

Four bugs inside the “verified” code

Vuln	Description	Algorithm
V10	Wrong rounding constant	ML-KEM
V11	Missing inverse NTT	ML-KEM
V12	False serialization bound	ML-KEM
V13	x^2 instead of $x \cdot y$	ML-DSA

- The deployed Rust implementations are currently correct—the risk is **indirect**
- A developer who trusts the spec and refactors to match it would silently introduce the errors
- The proofs are **not checked in any CI pipeline**

Section 13

Systemic Issues

The ADMIT_MODULES pattern

From the ML-KEM extraction Makefile:

```
ADMIT_MODULES = Libcrux_ml_kem.Ind_cpa.fst \  
  Libcrux_ml_kem.Sampling.fst \  
  Libcrux_ml_kem.Vector.Neon.Arithmetic.fst \  
  Libcrux_ml_kem.Vector.Neon.Compress.fst \  
  ...
```

- These modules are **admitted by default**: proofs are not checked
- “Lax checking” means the F^* type-checker accepts the code, but proofs are **not verified**
- Like spell-checking a math proof: syntax is fine, content may be wrong

Admitted proofs as systemic risk

Admitted proofs are **more dangerous** than unverified code

Unverified code (V1–V9):

- Was never claimed to be verified
- Developers and users can identify it as a risk

Admitted proofs (V10–V13):

- Code *is* claimed to be verified
- But the proofs are **silently not checked**
- Creates false confidence in the verified core
- A developer who trusts the spec and refactors the implementation to match it would **introduce the spec's errors into production**

What “lax checking” actually means

Level	Syntax	Types	Proofs
Lax checking	✓	✓	–
Runtime safety	✓	✓	Partial
Full verification	✓	✓	✓

- Lax checking confirms the code is **well-formed** $F^\star$
- It does **not** confirm that the proofs are valid
- A false lemma claiming $2^{12} < 2^1$ passes lax checking
- The NEON backend has **only** lax checking for all modules

Section 14

Comparative Analysis

How AWS does it differently

The same failure types exist in AWS's verified libcrypto—but they are managed differently

AWS verification projects:

- aws-lc-verification (SAW/Cryptol)
- s2n-bignum (HOL Light)
- mlkem-native (CBMC)

Same verification boundary challenges, **very different practices**

What good boundary management looks like

Practice	Cryspen	AWS
Soundness documents	Manual/stale	Structured
Proofs in CI	Not triggered	Every commit
Admitted modules documented	In Makefile only	Explicit
Verification scope communicated	Unqualified	Qualified
Axioms tested/proved	Neither	Tested

- AWS's soundness documents enumerate: verified properties, assumptions, residual risks, and open gaps
- The gold standard remains **seL4**: dedicated page listing every trust assumption

Parallels with CompCert

Yang et al.'s Csmith campaign (2011)

- Randomly generated C programs to find compiler bugs
- **Found zero wrong-code bugs** in CompCert's verified middle-end
- **Did find bugs** in the unverified front-end

Direct parallel to our findings:

- libcrux's verified core is more robust than its wrappers
- But we go further: the verified core can contain **specification errors** (V10, V11), **false proofs** (V12), and **false axioms** (V13) when the build system doesn't check them

Thompson's "Reflections on Trusting Trust"

"Who verifies the verifiers?"

- Ken Thompson (1984): You can't trust code you didn't write yourself
- Remains relevant: proofs dispatched through **unverified SMT solvers**
- F^* relies on Z3 for proof obligations
- Z3 is a complex piece of software with its own bugs
- The trusted computing base is **never zero**

*But we can at least ensure the proofs are **actually run***

Section 15

Recommendations and Lessons

Five recommendations

1. **Machine-readable verification manifests:** Every release should include a manifest of which code paths are verified, against which properties, for which platforms. Generated from build system, not manually maintained.
2. **Proofs in CI on every commit:** All proof obligations discharged in CI. Too-slow proofs flagged as incomplete, not silently admitted.
3. **No lax-mode escape hatches:** Proof tactics should fail loudly in lax mode, not silently admit all goals.

Five recommendations (continued)

4. **Clear scope communication:** Claims of “formally verified” should specify which properties, which code paths, which platforms. Unqualified claims should be reserved for systems with minimal TCB (CompCert, seL4 level).
5. **Defense in depth:** Formal verification is one component of a defense-in-depth strategy. Code review, specification compliance auditing, cross-platform testing, and fuzzing catch categories of bugs that formal verification does not.

Traditional practices remain essential alongside formal methods

Disclosure: A cautionary tale

- Feb 5, 2026: We submit PRs with tested fixes for V3–V6
- Feb 5–6: Cryspen **blocks our GitHub account**, closes all PRs
- Feb 9: Cryspen merges our fixes **without attribution**
- Feb 12: Cryspen publishes response stating “no bugs have been found in the verified code”
- Feb 17: We disclose V8–V9 (FIPS 204 violations)
- Mar 4: Cryspen merges fixes for V8–V9, crediting the discoverer

“No bugs in the verified code” is directly contradicted by V10–V13

Why this matters: The trust chain

Users trust the library → Library trusts the proofs → Proofs trust the spec

- If the spec is wrong (V10, V11): the entire chain is broken
- If the proofs are false (V12): the chain is illusory
- If axioms are wrong (V13): the chain is unsound
- If proofs are never checked: the chain doesn't exist

Trust is only as strong as the weakest link

Connecting Part 1 and Part 2

Part 1 showed us the **promise** of CAC

Part 2 shows us the **pitfalls**

- The same F[★] technology behind HACL[★] (success story) powers libcrux
- The difference is not the tools—it's the **engineering practices**
- EverCrypt documents its trust assumptions; libcrux does not
- HACL[★] ran full proofs in CI; libcrux admits modules by default

Verification is only as strong as the process around it

Auditing methodology

A lightweight methodology for detecting verification boundary failures

1. **Map the verification boundary:** Find extraction annotations, build system admission flags, unverified wrappers, platform-specific code paths
2. **Audit specifications against standards:** Check constants, domain transformations, required validations against FIPS/RFC
3. **Verify proofs are actually checked:** Run the full verifier, not just lax checking; look for `tadmit` and `assume`
4. **Cross-reference documentation:** Compare claimed verification status against actual build system behavior

What “formally verified” should mean

A spectrum of claims

1. **“Lax-checked”**: Syntax and types are valid (minimal guarantee)
2. **“Partially verified”**: Some modules have completed proofs, others are admitted
3. **“Verified with caveats”**: All proofs checked, but with documented trust assumptions (compiler, hardware, axioms)
4. **“Fully verified”**: Minimal TCB, all proofs in CI, explicit boundary documentation (CompCert/seL4 level)

Most projects are at level 2 or 3. Claims should reflect this.

Verification theatre, defined

Verification theatre is the practice of deploying formal methods in a manner that:

1. Establishes machine-checked proofs for a **subset** of code and properties, where some proofs may themselves be **unchecked, vacuous, or built on false axioms**
2. Communicates these proofs using language that implies **comprehensive coverage**
3. Fails to adequately disclose the verification boundary, unverified components, unchecked proofs, or properties not covered

Echoes Schneier's "security theater": an appearance of security that doesn't match reality

Looking forward: Post-quantum transition

The stakes are rising

- ML-KEM and ML-DSA are the **new NIST standards** for post-quantum cryptography
- They will be deployed in virtually every secure communication system
- Getting their implementations right is **critical**
- libcrux provides the ML-KEM implementation used by Google

The post-quantum transition is the moment when verification boundary practices matter most

The bottom line

Formal verification is genuinely valuable

- Machine-checked proofs of correctness, memory safety, and secret independence are real contributions to software security
- HACLS*, Fiat Crypto, Jasmin, EverCrypt: these are genuine achievements

But partial verification $\neq$ comprehensive assurance

- Treat verification boundaries as first-class documentation
- Ensure build systems actually verify the proofs they claim to check
- Formal methods **complement**, rather than replace, engineering practices

Verify the verifiers.



Applied Cryptography

Summer 2026

Part 2: Real-World Cryptography

2.5: High-Assurance Cryptography

Nadim Kobeissi

<https://appliedcryptography.page>